

BBMODEL Integration Guide

Goal: prepare a `.bbmodel` that is compatible with player skin application through ModelEngine 4.0.9.

Technical Pipeline

Execution flow for applying a skin to an animated model:

1. Retrieve the `PlayerProfile` (synchronously from the connected player, or through `SkinService` asynchronously for a third-party username).
2. Create a `Dummy<PlayerProfile>` and register it with ModelEngine (`createModeledEntity`).
3. Load the `ActiveModel` matching the `modelId` and attach it with `addModel(...)`.
4. Iterate through `activeModel.getBones()`; for each bone exposing a `PlayerLimb` behavior, call `setTexture(profile)`.
5. Start the animation through `activeModel.getAnimationHandler().playAnimation(animationName, ...)`.

`Dummy<PlayerProfile>` is the source of truth for the skin on the server side.
If `setTexture` is called correctly on each limb but the model still does not render, the issue is in the `.bbmodel` or the client resource pack, not in the Java code.

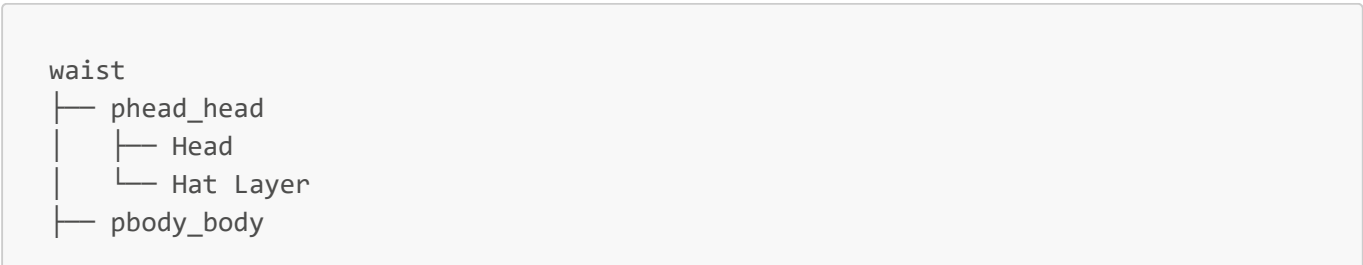
Preparing the `.bbmodel`

Bone naming (required)

Top-level bones must use exactly these names so that ModelEngine detects `PlayerLimb` behaviors:

Exact bone	Body part
<code>phead_head</code>	Head
<code>pbody_body</code>	Body
<code>prarm_right_arm</code>	Right arm
<code>plarm_left_arm</code>	Left arm
<code>prleg_right_leg</code>	Right leg
<code>plleg_left_leg</code>	Left leg

Expected hierarchy in Blockbench (only the first level matters for detection):



```

|   |   | Body
|   |   | Body Layer
|   |   |
|   |   | prarm_right_arm
|   |   |   | Right_arm
|   |   |   | Right Arm Layer
|   |   |
|   |   | plarm_left_arm
|   |   |   | Left_arm
|   |   |   | Left Arm Layer
|   |   |
|   |   | prleg_right_leg
|   |   |   | Right_leg
|   |   |   | Right Leg Layer
|   |   |
|   |   | plleg_left_leg
|   |   |   | Left_leg
|   |   |   | Left Leg Layer

```

Animation name (required)

The animation must be named exactly **dance** in Blockbench. The `animationName: dance` entry in `config.yml` must match that name.

Geometry rules

- Each limb must have visible cubes and must not be placed in a hidden group.
- Each limb must be independent (not parented to the head or to another limb).

ModelEngine Integration

1. Export the `.bbmodel` from Blockbench (keep textures and animations).
2. Copy the file into `plugins/ModelEngine/blueprints/`.
3. Verify on the server side that `createActiveModel(modelId)` returns a non-null `ActiveModel`.
4. Run `/dance debug` and verify that the `PlayerLimb` bones are detected (`Bones found: N`) and that `✓ Skin applied` appears for each expected limb.
5. Verify that the `dance` animation exists in the blueprint (visible in debug logs if the name does not match).

Quick Check

Server side (`/dance debug` logs)

- `Bones found: N` - non-zero value
- `✓ Skin applied` for each expected limb
- `activeModel != null`

Model / client side

- The Blockbench animation is named `dance` (and `animationName: dance` in `config.yml`)
- The limb cubes are present and visible in Blockbench
- The ModelEngine resource pack is correctly loaded by the client

Troubleshooting

Recommended isolation procedure

Create a minimal `.bbmodel` with only `phead_`, then add the limbs back one at a time and test at each step:

1. `phead_` - confirm that the head shows the skin
2. `pbody_` - confirm that the body shows the skin
3. `prarm_` / `plarm_` - right arm, then left arm
4. `prleg_` / `plleg_` - legs

Technical notes

- `HeadForcedImpl` is an internal behavior added automatically by ModelEngine; do not try to remove it from the plugin.
- If the head works but not the other limbs, the problem is in the `.bbmodel` geometry or the client resource pack, not in the Java code.
- `setTexture(PlayerProfile)` is the main approach; `setTexture(Player)` is implemented as a reflection-based fallback.

If the problem persists

Attach the `.bbmodel` and the `/danse debug` logs to an issue for analysis.